

Informatique scientifique 3

De l'ordinateur à la programmation d'un outil

Christophe Airiau

Septembre 2023

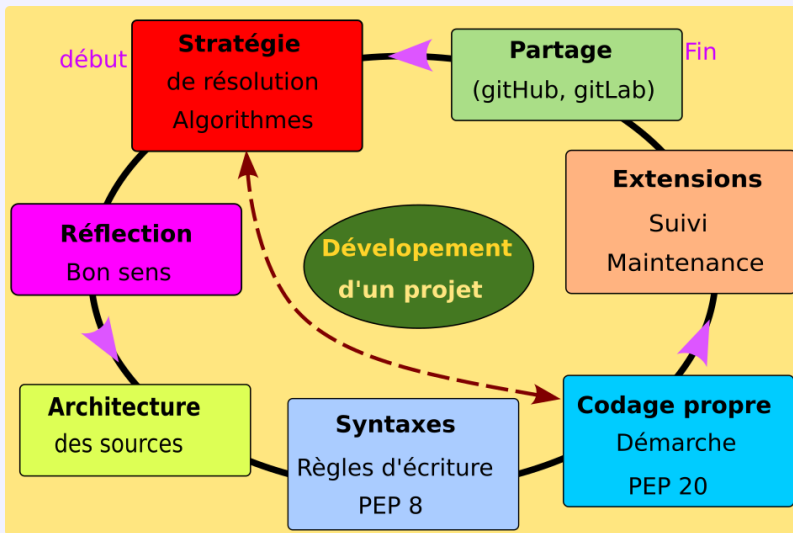


Sommaire

- 1 Mener un projet
 - Résoudre un problème

- Architecture d'un projet
- Maintenance
- Règles pour le codage

Développement de code : ingrédients



(utilisation du commentaire #)

- 1 Définir la structure des données à implémenter (représentation interne des données)
- 2 Inventorier les différents scénarios/configurations possibles
- 3 Faire des schémas, ordinogrammes pour traduire ses pensées
- 4 Définir les tâches et sous-tâches et donc les sous-algorithmes (entrées et sorties) et les hiérarchiser les uns par rapport au autre (qui appelle qui, combien de fois, où, quand, ...)
- 5 Écrire un squelette du programme qui contient les noms des variables et des fonctions vides
- 6 Écrire précisément le programme principal et les différentes fonctions.

- Penser dès le début aux différentes possibilités de structure et fixer les choix
- Différencier les étapes en décomposant en blocs d'instructions
- Les blocs permettent d'écrire un squelette simple :
 - lecture des paramètres
 - initialisation, pré-traitement
 - traitement des données et calcul des résultats
 - affichage et / ou sauvegarde des résultats.

Développement progressif

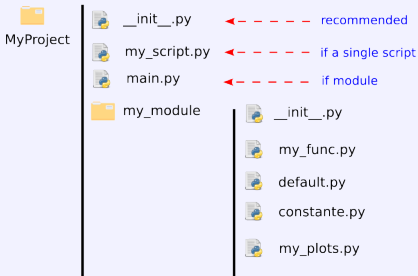
Coder 5 lignes, tester, corriger puis continuer.

- Donner des noms **évocateurs** pour tous : variables, constantes, fonctions, ...
- Introduire une seule tâche par fonctions, notion de blocs d'instructions
- Documenter (Ajouter des commentaires au moment du codage (et pas après ...))
- Ajouter des affichages (temporaires ou permanents) pour vérifier les calculs et valeurs intermédiaires
- Résoudre morceau par morceau, en validant par des essais appropriés à chaque tâche
- Explorer tous les cheminements possibles à l'intérieur de l'algorithme
- Construire des tests représentatifs de l'ensemble des cas possibles, du cas général aux cas extrêmes ou exceptionnels

- toujours avoir **une idée précise du travail à accomplir** : identifier les entrées, les sorties, la méthode pour passer de l'un à l'autre.
- procéder pas par pas
- introduire des variables ou paramètres qu'on initialise (pas de valeurs numériques au milieu du code)
- correction des messages d'erreurs
- ajouter des commentaires pour la documentation
- tester sur des cas simples
- augmenter la difficulté progressivement
- ne pas hésiter à ré-écrire le code une fois le travail effectué
- **S'informer, regarder des exemples, comprendre, apprendre, pratiquer.**

La manière de résoudre le problème et d'écrire le code est aussi importante que d'obtenir un bon résultat.

- Un script simple si `ligne_de_code <= 300` (avec des fonctions)
- Un script principal `main.py` très court et un (ou plusieurs) module
- Le module contient un ou plusieurs fichiers avec des thèmes bien distincts, éventuellement des classes
- Choisir des noms de fichiers ou modules parlant



- toujours créer le fichier vide `__init__.py`
- définir les paramètres par défauts et les constantes
- séparer si possible, le calcul des graphiques
- on peut faire plus complet en dissociant les sources dans un dossier `src` et créer une documentation dans `doc` juste sous `MyProject`
- le choix de l'IDE ne doit pas modifier l'architecture.

👉 L'architecture d'un projet doit aussi contenir d'autres fichiers d'informations ou de configuration, ou d'installation.

Voir des exemples sur github.com

Extensions, maintenance d'un code

- La maintenance est souvent facile si le code respecte les règles de l'art
- Enrichir (étendre) les fonctionnalités progressivement (ajout de nouveaux paramètres, variables, fonctions)
- Penser en terme de boîte à outils, bibliothèque de composants logiciels réutilisables
- Faciliter la maintenance du code
 - écrire des choses simples et compréhensibles par tous
 - modifier tel ou tel traitement sans changer le rôle de la fonction ou du bloc considéré
 - Commenter, penser au partage et à la diffusion de l'oeuvre

Norme d'écriture d'un code : PEP

- Si l'algorithme est bien pensé et écrit, le passage à l'écriture du programme qui fonctionne pourra être très rapide.
- Il existe des règles (ou recommandations fortes) pour écrire un programme efficace, claire, facile à comprendre, maintenir, partager, modifier.
- En python c'est la "norme" PEP 8 (Python Enhancement Proposals)
- **Autant apprendre les bonnes manières tout de suite !**
- Description complète de la PEP 8 :
<https://peps.python.org/pep-0008/>
- Il existe d'autres PEP : <https://peps.python.org/pep-0000/>
Dans les faits, on ne la suit pas toujours rigoureusement mais **les notations doivent rester cohérentes dans un même projet.**

PEP 8 : 1 - INDENTATION

```
1 class Machine(objects):
2     """ indenter de 4 espaces"""
3     def __init__(self):
4         self.value = 10 # 4
5     espaces
6
7 def myfunc()
8     """ indenter de 4 espaces"""
9     if bidule == machin:
10         print("zut") # 4 espaces
11     for i in range(10):
12         print(i) # 4 espaces
13     for i in range(20):
14         print(i) # 4 espaces
15
```

- 4 espaces pour les indentations
- remplacer les tabulations par 4 espaces
- erreurs si on mélange tabulations et espaces d'indentation
- erreurs si on ne respecte pas l'indentation constante

PEP 8 : 2 - LONGUEUR D'UNE LIGNE

```
8 d = {"un": [1, 4, 8, 9], "deux": "une longue phrase", "trois": [120, 300, 400],
9      "quatre": 100}
10 # autre possibilité
11 d1 = {"un": [1, 4, 8, 9],
12       "deux": "une longue phrase",
13       "trois": [120, 300, 400],
14       "quatre": 100
15     }
16 print("il faut respecter les recommandations de la norme PEP 8, sinon" +
17       "le code ne sera pas transportable")
18
19 for nom, voiture, avion in zip(liste_noms, liste_voitures, liste_avions):
20     print("nom: {:10s}, voiture: {:15s}, avions: {20s}".format(nom, voiture,
21                                                                avion))
22 # utiliser le \ pour indiquer la continuation de la ligne
23 if cond1 == 0 and cond2 < 10 and cond3 == "un long mot" and \
24     couleur in ["rouge", "vert"]:
25     pass
```

- ne pas dépasser 79 caractères
- sinon couper la ligne avec cohérence
- aligner les éléments
- utiliser le symbole de continuation de ligne \
- les IDE aide à respecter cela en indiquant la fin de ligne.
- **encoder en UTF-8**

PEP 8 : 3 - LIGNE VIDE

On ne met pas des lignes vides partout et quand on veut !

Cas	nombre de lignes vide
Entre deux classes	2
Entre deux méthodes (dans une classe)	1
Entre deux fonctions (dans un script)	2
Avant et après les imports	1

- Dans une fonction, ou dans le corps du code, on ne saute des lignes qu'avec parcimonie pour mettre en évidence des blocs de code.
- Des IDE (Pycharm, VSCode) aident à respecter cela parfois.

PEP 8 : 4 - IMPORTS

- Un import par ligne
- On évite `from numpy import *`, sauf si on a vraiment besoin de tout ce qui est dans le module (générique ou personnel)
- à la place du `*` on indique les méthodes utilisées.
- Positioner les imports en début de fichier, après le commentaire sur le jeu de caractères et les commentaires sur le contenu du module ou du script
- Mettre dans l'ordre, les bibliothèques standards, les modules tiers et enfin les modules locaux (en général celui du développeur)
- Pour les imports classiques, conserver les raccourcis habituels (np, plt, pd pour panda, ...)

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Commentaire sur mon module ou script,
5  name: mymodule : perform simulation of a model
6  @author: C. Airiau, Toulouse III university
7  """
8
9  import numpy as np
10 import matplotlib.pyplot as plt
11 from mymodule import myfunction, mything, myvariable
12
13 # suite du code ...
14
```

PEP 8 : 5 - LES ESPACES

```

1  a = 2 * b
2  b *= 2
3  c, b = b, a
4  if x == 2:
5      pass
6
7  def func(x, y, z=1)
8      return sum(x, y, z)
9
10 z = func(1, 2, z=3)
11 d = dic(a=1, b=2)
12 d = {"a": 1, "b": 2}
13 tu = (1, 2, "m",)      # no space
14                          before )
15
16 for x == 2: print(x)    # to avoid
17 for x == 2:
18     print(x)            #
19                          recommended
20
21 i = i + 1
22 j += 1                  # it is
23                          better
24
25 x = x*2 - 1             # not
26                          possible for 2*x - 1
27
28 hypot2 = x*x + y*y      # just for +
29 c = (a+b) * (a-b)       # just for *

```

- mettre un espace entre chaque opérateur (*, +, -, /, and, or, ==, not, ...), mais par groupement seulement
- un espace après un point ou une virgule
- pas d'espace dans le = des mots clés des fonctions, ni dans les dictionnaires
- 4 espaces pour l'indentation

PEP 8 : 6 - LES COMMENTAIRES

```
1      # To calculate an angle from degrees to in radians
2      ang = np.deg2rad(54)
3      cosa = r * np.cos(ang)      # x component
4      ang += 10                    # rotation of 10 degrees
5      pass
6      # my algorithm is as followed:
7      #     1. action 1: ...
8      #     2. action 2: ...
9      # I am doing nothing          # unuseful, to delete
10
```

- le commentaire ne doit pas contredire le code dessous (mettre à jour)
- mettre des phrases, ou des verbes pour des actions
- commencer le commentaire par une lettre majuscule
- chaque ligne de commentaire commence par #
- commentaire après du code, sur la même ligne : # et les alignés
- écrire en anglais (pour les futurs utilisateurs ou développeurs)
- uniquement des commentaires utiles, ne répétant pas le code.
- il existe un format spécifique pour créer des documentations automatiques de code

PEP 8 : 7 - LES CHAÎNES DE DOCUMENTATIONS - DOCSTRINGS

```
def myfunc(x, y):  
    """ to do the product x * y  
    Parameters:  
        x: float  
            meaning  
        y: float  
            meaning  
    Returns:  
        float: x * y  
    """  
    pass  
  
class myclass(objects):  
    """  
    This class treats ...  
    """  
    def __init__(self, params):  
        """ class initialization  
        """  
        pass
```

- Toujours en écrire au début d'un module (fichier) qui décrit rapidement le contenu, et souvent la liste des classes ou des fonctions du module.
- Toujours en mettre après la déclaration d'une fonction ou d'une classe
- Il existe un format spécifique pour créer de la documentation automatique (s'informer)
- La docstrings devient automatiquement l'attribut `__doc__` de l'objet
- On la retrouvera avec l'instruction `help(myfunc)` par exemple
- uniquement des commentaires utiles, ne répétant pas le code.
- On peut utiliser les docstrings pour mettre de long commentaires dans le code, mais elles n'apparaîtront pas avec la fonction `help`.

PEP 8 : 8 - CONVENTION DE NOMMAGE

- Il existe deux conventions et une notation spéciale :
 - snake_case : lettres minuscules + chiffres + "_"
 - CamelCase : lettres minuscules, majuscule au début ou pour séparer les mots.
 - "dunders" : deux soulignées en début et fin : `__init__`
- modules, fichiers : snake_case, noms courts (< 12 caractères par exemple)
- paquets : lettres minuscules (sans _)
- Noms des fonctions, variables (globales ou pas) : snake_case
- Constante : majuscules avec (ou pas) `__` en début et fin de nom dans les classes
- Noms compatibles format ASCII (pas d'accent, de caractères spéciaux sauf _)
- Éviter les lettres "l" (L miniscule), "O", "I" (i majuscule) qu'on peut confondre avec les chiffres 1, 0
- Pas de nom trop long, utiliser des raccourcis expressifs :
`young_modulus`, `car_mass`, `area`, `length`, ...
- dans les classes, nommer un attribut comme `_var_` indique que c'est une variable ou attribut local qui n'est pas destinée à sortir de la classe (non publique)
- sinon les attributs et méthodes de classes suivent le nommage snake_case
- Ne pas utiliser de noms réservés sinon le modifier légèrement :
`print_`, `prt`, `type_`, `len_` or `length`

PEP 8 : conclusion

Tout comme on ne peut pas bien utiliser une langue si on ne connaît pas l'orthographe, le vocabulaire ou la grammaire,

on ne peut pas bien apprendre un langage informatique sans connaître les instructions de base et les règles ou conventions d'écriture et d'utilisation.

PEP20 : règles 1, 2 et 3

1 - Beautiful is better than ugly. Le beau est préférable au laid.

Le point de départ. Un bon code Python se doit d'être beau, c'est-à-dire agréable à regarder (PEP8) : avoir une structure visible et claire.

2 - Explicit is better than implicit. L'explicite est préférable à l'implicite.

Un développeur doit pouvoir lire un code Python sans se demander sans cesse ce que fait telle ou telle ligne. Utiliser des noms et des constructions explicites permet de limiter ce genre de problèmes.

3 - Simple is better than complex. Le simple est préférable au complexe.

Certaines structures du langage vont s'avérer plus complexes que d'autres.

L'application d'un décorateur est une instruction complexe, par les mécanismes qu'elle met en oeuvre : patron de conception décorateur, fonctions passées implicitement en paramètre.

PEP20 : règles 4 et 5

4 - Complex is better than complicated. Le complexe est préférable au compliqué.

- « compliqué » se rapporte à l'utilisation. Un code compliqué est difficile à relire et à maintenir.
- Un code complexe utilise des mécanismes avancés, mais il peut être simple à appréhender.

5 - Flat is better than nested. Le plat est préférable à l'imbriqué.

- Il est facile de se perdre dans la lecture d'un code. Notamment s'il contient de nombreux niveaux d'imbrications.
- Produire du code plat chaque fois que cela est possible et éviter les imbrications inutile.
- Par exemple, pour une fonction, on retourne les valeurs (en quittant) dès que c'est possible.

PEP20 : règles 6 et 7

6 - Sparse is better than dense. L'aéré est préférable au dense.

- Un code compréhensible est un code aéré.
- La syntaxe même du langage se base sur l'indentation pour séparer les blocs logiques.

7 - Readability counts. La lisibilité compte.

- Un développeur passe plus de temps à lire son code qu'à l'écrire.
- Le code se doit donc d'être lisible facilement,
- La lisibilité passera par de nombreux points évoqués par les autres directives, dont le choix judicieux des noms de fonctions et variables

PEP20 : règles 8 et 9

8 - Special cases aren't special enough to break the rules. Les cas spéciaux ne le sont pas assez pour briser les règles.

- C'est le principe de la **cohérence**.
- Les mêmes règles s'appliquent pour tous, ce n'est pas parce qu'un bout de code semble sortir du lot qu'il y déroge.

9 - Although practicality beats purity. Bien que la praticité prévale sur la pureté.

- Et cette règle, qui nuance la précédente, représente le bon sens.
- Il peut devenir nécessaire d'outrepasser les règles pour des raisons pratiques, telles que des questions de performances.
- Cela doit dans tous les cas rester anecdotique.

PEP20 : règles 10 et 11

10 - Errors should never pass silently. Les erreurs ne devraient jamais se produire silencieusement.

- Quand une erreur se produit c'est qu'il y a un problème, quel qu'il soit.
- Ce problème ne doit jamais être masqué par le développeur.

11 - Unless explicitly silenced. À moins d'être explicitement tues.

- Le développeur peut ensuite choisir d'ignorer une erreur en particulier, car celle-ci est attendue dans ce cas précis.
- Mais il le fera de façon explicite, avec un bloc try/except par exemple.

PEP20 : règles 12 et 13

12 - In the face of ambiguity, refuse the temptation to guess. En cas d'ambiguïté, résister à la tentation de deviner.

- Deviner implique un choix, choix qui ne sera pas forcément clair pour tous les développeurs.
- S'il n'est pas clair, il faut le rendre explicite.

13 - There should be one – and preferably only one – obvious way to do it. Il devrait y avoir une – et de préférence une seule – manière évidente de le faire.

- Python prône le fait qu'il existe toujours une manière optimale de procéder,
- Toutes les solutions ne se valent pas.
- La solution la plus évidente est préférable.

PEP20 : règles 14 et 15

14 - Although that way may not be obvious at first unless you're Dutch. Bien que cette manière ne vous semble pas évidente au premier abord, à moins que vous ne soyez néerlandais.

- Cependant, l'évidence n'est pas innée. Elle vient avec la pratique, et est entre autres dictée par cette PEP.
- La manière évidente est la manière la plus **idiomatique**.
- Cette règle se termine par une note humoristique sur Guido van Rossum, néerlandais, le créateur de Python.

15 - Now is better than never. Maintenant est préférable à jamais.

- La procrastination est l'ennemie du développeur Python.
- Si vous avez besoin d'une fonctionnalité manquante, implémentez-la,
- Ne codez pas de rustines temporaires pour y revenir « plus tard ».

PEP20 : règles 16 et 17

16 - Although never is often better than right now. Bien que jamais soit souvent préférable à tout de suite.

- S'assurer cependant que cette fonctionnalité soit vraiment nécessaire.
- Sinon, il peut être préférable de remettre à plus tard.

17 - If the implementation is hard to explain, it's a bad idea. Si l'implémentation est difficile à expliquer, c'est une mauvaise idée.

- Une implémentation difficile à expliquer est compliquée, elle produira du code compliqué.
- On préférera donc l'éviter au profit d'une implémentation plus simple, plus facile à expliquer.

PEP20 : règles 18 et 19

18 - If the implementation is easy to explain, it may be a good idea. Si l'implémentation est facile à expliquer, il peut s'agir d'une bonne idée.

- Pour autant, être facile à expliquer n'en fait pas une bonne implémentation.
- C'est une bonne chose, mais ce n'est pas un critère suffisant.

19 - Namespaces are one honking great idea – let's do more of those ! Les espaces de noms sont une sacrée bonne idée – utilisons-les plus souvent ! .

- Les espaces de noms sont créés à l'aide des paquets, modules et objets
- ils permettent de diviser les classes et fonctions en ensembles logiques.
- Ils évitent aussi les conflits de noms, un même nom pouvant être utilisé pour des valeurs différentes dans des espaces différents.
- On aimera alors en user pour bien classer nos objets.

Quand un code doit-il être amélioré ?

- Des méthodes (fonctions) trop longues (au delà de 20 lignes)
- Des classes trop longues (au delà de 200 lignes)
- Une incapacité à lire le code "en diagonale"
(il faut réfléchir pour comprendre ce qui est écrit, les méthodes ...),
notamment après plusieurs mois...
- L'abus de méthodes statiques (préférer la POO)
- Plus de 2 niveaux d'indentation dans une méthode
- Des objets sans méthodes (en dehors des getters/setters en POO)

☞ Suivre son intuition si le code semble trop compliqué à lire ou à comprendre,
s'il n'est pas clair, c'est qu'il faut le simplifier, l'améliorer, le réorganiser

☞ **La manière de résoudre le problème et d'écrire le code est aussi importante que d'obtenir un bon résultat.**

CC : PAS DE NOMBRES MAGIQUES

BAD

```
1 x = [3, 5, -5 , 10]
2 y = []
3 for i in range(4):
4     y.append(x[i]**2)
5 z = y[1: 3]
6
7 w = np.linspace(0, 1, 51)
8 for i in range(51):
9     print(2*w + 1)
```

GOOD

```
1 x = [3, 5, -5 , 10]
2 for x in x:
3     y.append(x**2)
4 # better
5 y = [x**2 for x in x]
6 z = y[1:]
7
8 NPT = 51
9 w = np.linspace(0, 1, NPT)
10 for i in range(NPT):
11     print(2*w[i] + 1)
12 # better:
13 for x in x:
14     print( 2*w + 1)
```

- Il ne faut pas écrire des nombres "magiques" dans le code, car les valeurs peuvent changer si on modifie légèrement les entrées.
- Modifier la façon d'écrire ou ajouter des constantes avec une valeur prédéfinie.

CC : PAS DE DUPLICATION

BAD

```
1 x = [3, 5, -5, 10]
2 y = [eta**2 for eta in x]
3 x1 = [3, 5, -5, 10]
4 y1 = [eta**2 for eta in x1]
```

GOOD

```
1 def square(x):
2     """ calculate the square of x """
3     return [eta**2 for eta in x]
4
5 y, y1 = square(x), square(x1)
```

- Ne pas écrire deux fois la même chose, mais définir une fonction qui fait le travail
- En cas de modification, cela n'est fait qu'à un seul endroit.

CC : RESTER CONSISTENT

BAD

```
1 x = [3, 5, -5, 10]
2 y = [eta**2 for eta in x]
3 # same thing but without
  consistency
4 x_var = [3, 5, -5, 10]
5 y_var = []
6 for i in range(len(x_var)):
7     y_var.append(x_var[i]**2)
```

GOOD

```
1 def square(x):
2     """ calculate the square of x
3     """
4     return [eta**2 for eta in x]
5 y, y1 = square(x), square(x1)
```

Rester consistent :

- dans les choix des noms,
- dans sa façon de programmer.
- dans les algorithmes